
Module 14

Git, GitHub & Project Development

From first commit to a structured Django project

The finish line is a shippable change

- Flow 1 Track work with Git snapshots.
- Flow 2 Collaborate through GitHub branches and pull requests.
- Flow 3 Start, structure, and implement the main Django project.
- Outcome** A GitHub repository containing a runnable Django skeleton, one focused feature branch, and a reviewed merge into the default branch.

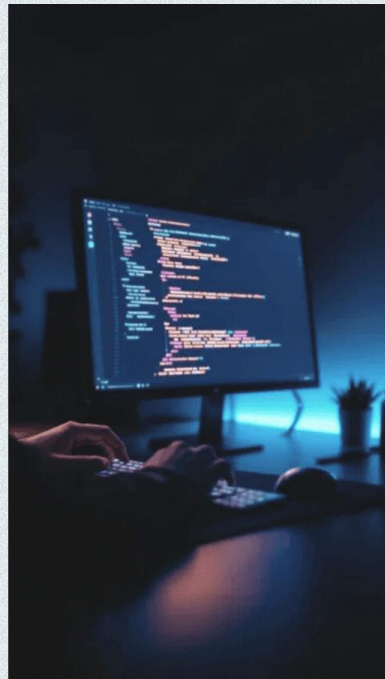


Git turns changes into recoverable snapshots

The mental model:

Working tree → git commit → git add → staging area →
→ local repository → git push → GitHub
remote

- ✎ The working tree is where you edit your files.
- ⊕ Staging selects the exact changes for the next snapshot.
- 🔒 A commit is a named checkpoint; always write what changed and why.
- 🔍 **Quick check:** Run git status before and after each checkpoint.



Build the first version before adding adding features

```
mkdir campus-portal && cd campus-portal  
git init  
git add .  
git commit -m "chore: start project"
```



Logical Changes

Make one logical change at a time to keep history clean.



Small Commits

Keep commits small enough to explain in one sentence.



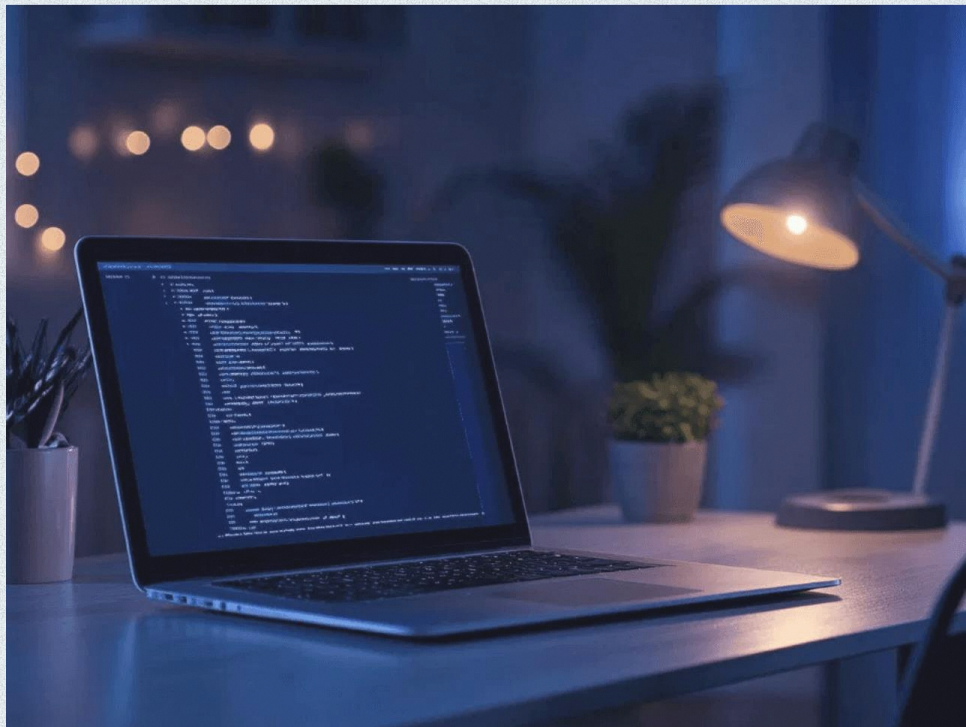
Clear History

Use `git log --oneline` to see the story of your project.



Checkpoint

Can a classmate clone the repository and understand its purpose from the README?



Push shares work; pull brings the team back together

Rule: Pull before you start; push when your checkpoint is useful. is useful. Connecting your local repository to GitHub enables the daily sync loop.



1. Connect Local to Remote

Link your repository and push the initial state:

```
git remote add origin <url>  
git push -u origin main
```



2. Pull to Sync

```
git pull --rebase origin main
```

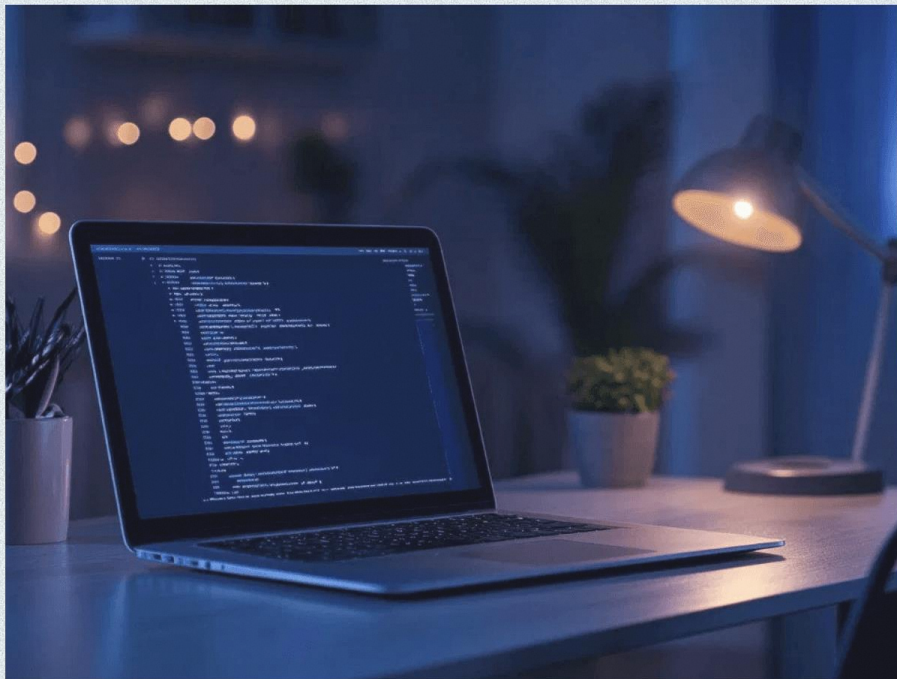
Always start your work session with
to fetch your team's latest updates.



3. Push to Share

```
git push origin <branch>
```

After working locally and committing, use
to share your checkpoints.



Branches isolate risk without copying the project

A branch is a lightweight pointer to a commit, allowing students to experiment safely without changing the default branch.

🔑 Use a branch for one outcome

Create: `git switch -c feature/homepage` or
`git checkout -b feature/homepage`
Push: `git push -u origin feature/homepage`
`feature/homepage`

📁 Good branch names

`fix/login-redirect`

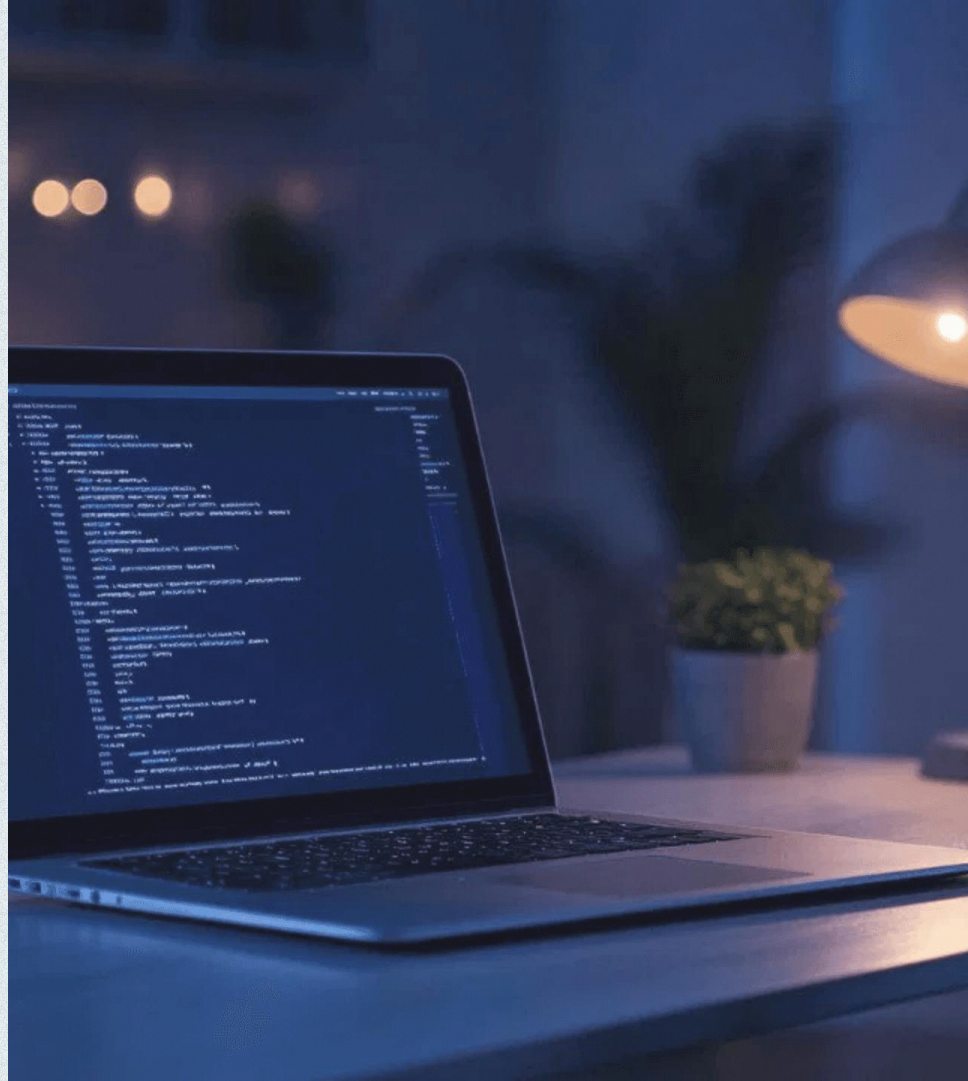
Use descriptive prefixes:

`docs/setup`

`feature/homepage`

⚡ Why it works

Unlike older systems that copy files, a Git branch is just a 41-byte file pointing to a commit, making creation instantaneous.



GitHub flow makes review part of development

The repeatable loop: Issue → branch → small commits → push → pull
→ pull request → review/checks → merge → pull latest main

🎯 State the Problem

Clearly explain the issue being addressed and the intended behavior of behavior of your changes.

🔧 Test Instructions

Provide clear steps on how reviewers can test and verify the changes changes locally.

💬 Focused Review

Keep the code diff focused on one feature and actively respond to review comments.

👥 Teaching Move

Pair students: one author, one reviewer; swap roles on the next branch to build empathy.



Start Django with a clean, repeatable bootstrap

From empty folder to running project. A clean bootstrap ensures your environment is isolated and your project structure is ready for version control.



1. Virtual Environment

```
python -m venv .venv  
source .venv/bin/activate
```



2. Install & Start

```
python -m pip install django  
django-admin startproject config .
```



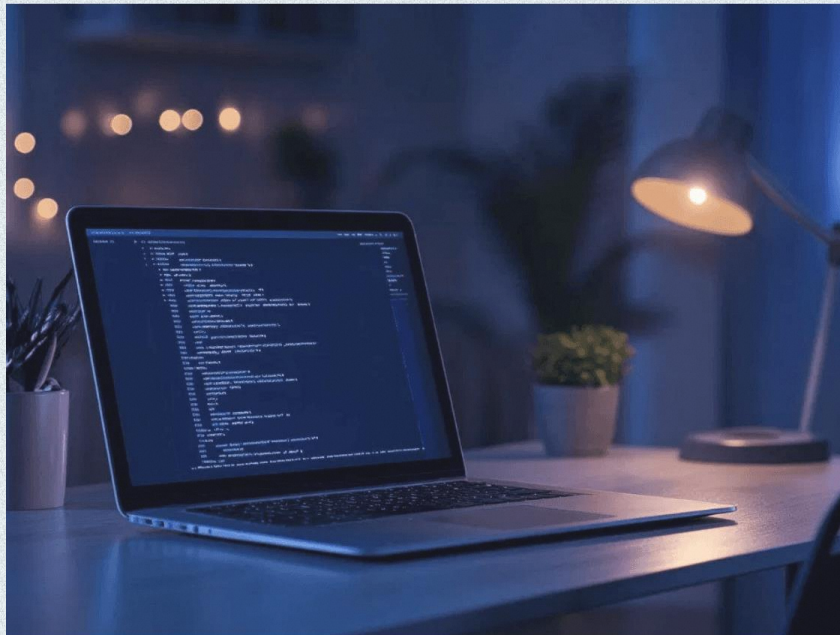
3. First Commit

Include `manage.py` , `config/` , `.gitignore` , and `README.md` .



4. Checkpoint

Run `python manage.py runserver` , open `http://127.0.0.1:8000/` , and commit the runnable skeleton.



Separate project configuration from app behavior



```
campus-portal/  
├── manage.py  
├── config/      # settings, URLs  
└── portal/      # focused Django app  
    ├── models.py  
    ├── views.py  
    └── urls.py
```



The Project

A collection of configuration and apps for a particular website.



The App

A web application that does something specific (e.g., a blog system, a poll system, a poll app).

Key distinction: A project is the configuration plus its apps. An app owns one focused capability and can be reused across multiple projects.

Use the time to repeat the loop

Follow this structured timeline to progress from local version control to a fully collaborative Django feature.

- 🕒 **0:00–0:45:** Git model, repository, and first commit
- 🔗 **0:45–2:15:** GitHub remote, branches, pull requests, and peer review review
- 💻 **2:15–3:15:** Bootstrap and run the Django project
- ✓ **3:15–4:00:** Implement one feature, merge, and reflect

Leave with this habit: Make the smallest useful change → checkpoint it → share it → review it → integrate it.

